

Universidad del Valle de Guatemala

Curso: Lógica Matemática

Docente: Paulo Mejía

Proyecto #1

Ecuaciones de recurrencia y complejidad algorítmica

Integrantes

Diego Quixchan
Denil Parada
Adrián Penagos
Harry Mendez
Juan Gualim
Wilson Peña

12 de agosto de 2026

Índice

Introducción	3
1. Complejidad $\mathcal{O}(\lg n)$: búsqueda binaria	3
1.1. Implementación	3
1.2. Recurrencia y resolución	3
1.3. Evidencia experimental	4
2. Complejidad $\mathcal{O}(n)$: máximo y mínimo	5
2.1. Implementación	5
2.2. Recurrencia y resolución	5
2.3. Evidencia experimental	6
3. Complejidad $\mathcal{O}(n \lg n)$: Merge Sort	7
3.1. Implementación	7
3.2. Recurrencia y resolución	7
3.3. Evidencia experimental	8
4. Complejidad $\mathcal{O}(n^2)$: trabajo lineal por nivel	8
4.1. Implementación	8
4.2. Recurrencia y resolución	9
4.3. Evidencia experimental	9
5. Complejidad $\mathcal{O}(n^3)$: trabajo cuadrático por nivel	10
5.1. Implementación	10
5.2. Recurrencia y resolución	10
5.3. Evidencia experimental	11
6. Complejidad $\mathcal{O}(n^{10})$: trabajo n^9 por nivel	12
6.1. Implementación	12
6.2. Recurrencia y resolución	12
6.3. Evidencia experimental	13
7. Complejidad $\mathcal{O}(n!)$: permutaciones	13
7.1. Implementación	13
7.2. Recurrencia y resolución	14
7.3. Evidencia experimental	14

8. Complejidad $\mathcal{O}(2^n)$: Torres de Hanoi	15
8.1. Implementación	15
8.2. Recurrencia y resolución	16
8.3. Evidencia experimental	16
9. Conclusiones	17

Introducción

Las relaciones de recurrencia permiten describir el costo de un algoritmo recursivo en función del costo de resolver instancias más pequeñas del mismo problema. En este proyecto se implementaron en Java ocho algoritmos o procedimientos con comportamientos representativos de las complejidades $\mathcal{O}(\lg n)$, $\mathcal{O}(n)$, $\mathcal{O}(n \lg n)$, $\mathcal{O}(n^2)$, $\mathcal{O}(n^3)$, $\mathcal{O}(n^{10})$, $\mathcal{O}(n!)$ y $\mathcal{O}(2^n)$. Para cada caso se contabilizaron operaciones relevantes, se realizaron pruebas con distintos valores de n y se compararon los resultados experimentales con la solución de su relación de recurrencia.

La notación $\lg n$ se utiliza para $\log_2 n$. Las resoluciones emplean los métodos estudiados en el curso: sustitución, relaciones lineales homogéneas y no homogéneas, divide y vencerás y Teorema Maestro, según corresponda. El propósito de las tablas no es medir tiempo de ejecución, sino evidenciar directamente el crecimiento de las operaciones asociadas a cada recurrencia.

1. Complejidad $\mathcal{O}(\lg n)$: búsqueda binaria

1.1. Implementación

La búsqueda binaria trabaja sobre un arreglo ordenado y, en cada llamada, conserva únicamente una mitad del intervalo. La implementación recursiva esencial es:

```
int buscar(int[] a, int x, int izq, int der) {
    if (izq > der) return -1;
    int medio = (izq + der) / 2;
    // se contabilizan las comparaciones definidas para el experimento
    if (a[medio] == x) return medio;
    if (x < a[medio]) return buscar(a, x, izq, medio - 1);
    return buscar(a, x, medio + 1, der);
}
```

El contador experimental considera dos comparaciones por nivel y una comparación en el caso final, por lo que $T(1) = 1$.

1.2. Recurrencia y resolución

El contador satisface

$$T(n) = T\left(\frac{n}{2}\right) + 2, \quad T(1) = 1.$$

Esta es una recurrencia de divide y vencerás de la forma

$$f(n) = af\left(\frac{n}{b}\right) + c,$$

con $a = 1$, $b = 2$ y $c = 2$. El teorema visto en clase establece que, cuando $a = 1$, el crecimiento es $\mathcal{O}(\log_b n)$; por tanto,

$$T(n) \in \mathcal{O}(\log_2 n) = \mathcal{O}(\lg n).$$

Para obtener además la expresión exacta, se aplica sustitución sucesiva:

$$\begin{aligned} T(n) &= T(n/2) + 2, \\ &= T(n/4) + 4, \\ &= T(n/8) + 6, \\ &\vdots \\ &= T\left(\frac{n}{2^k}\right) + 2k. \end{aligned}$$

La recursión termina cuando $n/2^k = 1$, es decir, $n = 2^k$ y

$$k = \log_2 n = \lg n.$$

Sustituyendo el caso base $T(1) = 1$:

$$T(n) = 1 + 2 \lg n = \boxed{2 \lg n + 1}.$$

El término dominante es $\lg n$, por lo que finalmente

$$\boxed{T(n) \in \mathcal{O}(\lg n)}.$$

1.3. Evidencia experimental

n	Comparaciones	$\lg n$	$2 \lg n + 1$
16	9	4	9
32	11	5	11
64	13	6	13
128	15	7	15
256	17	8	17
512	19	9	19
1024	21	10	21

Cuadro 1: Búsqueda binaria: resultados experimentales.

Los valores experimentales coinciden exactamente con $2 \lg n + 1$.

```
=====
BUSQUEDA BINARIA RECURSIVA - O(lg n)
=====
```

n	Elemento	Encontrado	Posicion	Comparaciones	log2(n)

16	15	Si	15	9	4
32	31	Si	31	11	5
64	63	Si	63	13	6
128	127	Si	127	15	7
256	255	Si	255	17	8
512	511	Si	511	19	9
1024	1023	Si	1023	21	10

```
=====
```

Figura 1: Ejecución de la búsqueda binaria recursiva.

2. Complejidad $\mathcal{O}(n)$: máximo y mínimo

2.1. Implementación

El arreglo se divide en dos mitades; ambas se resuelven recursivamente y sus resultados se combinan con una comparación de mínimos y otra de máximos.

```
Resultado resolver(int[] a, int izq, int der) {
    if (izq == der) return new Resultado(a[izq], a[izq]);
    int m = (izq + der) / 2;
    Resultado i = resolver(a, izq, m);
    Resultado d = resolver(a, m + 1, der);
    comparaciones += 2;
    return new Resultado(Math.min(i.min, d.min), Math.max(i.max, d.max));
}
```

2.2. Recurrencia y resolución

La implementación genera dos subproblemas de tamaño $n/2$ y realiza dos comparaciones al combinar sus resultados:

$$T(n) = 2T\left(\frac{n}{2}\right) + 2, \quad T(1) = 0.$$

Es de la forma $f(n) = af(n/b) + c$, con $a = 2$, $b = 2$ y $c = 2$. Como $a > 1$, el teorema de divide y vencerás estudiado en clase da

$$T(n) \in \mathcal{O}\left(n^{\log_b a}\right) = \mathcal{O}\left(n^{\log_2 2}\right) = \mathcal{O}(n).$$

Para encontrar la solución exacta se desarrolla la recurrencia:

$$\begin{aligned}
T(n) &= 2T(n/2) + 2, \\
&= 2[2T(n/4) + 2] + 2 = 2^2T(n/2^2) + 2(1 + 2), \\
&= 2^3T(n/2^3) + 2(1 + 2 + 2^2), \\
&\vdots \\
&= 2^kT(n/2^k) + 2\sum_{i=0}^{k-1} 2^i.
\end{aligned}$$

El caso base se alcanza cuando $n/2^k = 1$, por lo que $2^k = n$ y $k = \lg n$. Usando $T(1) = 0$ y la suma geométrica

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1,$$

se obtiene

$$T(n) = 2(2^k - 1) = 2(n - 1) = \boxed{2n - 2}.$$

Su término dominante es lineal, de modo que

$$\boxed{T(n) \in \mathcal{O}(n)}.$$

2.3. Evidencia experimental

n	Mínimo	Máximo	Comparaciones = $2(n - 1)$
16	0	15	30
32	0	31	62
64	0	63	126
128	0	127	254
256	0	255	510
512	0	511	1022
1024	0	1023	2046

Cuadro 2: Máximo y mínimo por divide y vencerás.

```

MAXIMO Y MINIMO POR DIVIDE Y VENCERAS - O(n)
=====
n      Minimo    Maximo    Comparaciones    2(n - 1)
-----
16      0           15        30              30
32      0           31        62              62
64      0           63       126             126
128     0          127       254             254
256     0          255       510             510
512     0          511      1022            1022
1024    0          1023     2046            2046
=====

```

Figura 2: Ejecución del algoritmo de máximo y mínimo por divide y vencerás.

3. Complejidad $\mathcal{O}(n \lg n)$: Merge Sort

3.1. Implementación

Merge Sort divide el arreglo en dos mitades, ordena cada una recursivamente y luego las mezcla. El trabajo de mezcla es lineal en el tamaño del segmento.

```
void mergeSort(int[] a, int izq, int der) {
    if (izq >= der) return;
    int m = (izq + der) / 2;
    mergeSort(a, izq, m);
    mergeSort(a, m + 1, der);
    merge(a, izq, m, der); // procesa linealmente el segmento
}
```

3.2. Recurrencia y resolución

Merge Sort resuelve dos subproblemas de tamaño $n/2$ y utiliza trabajo lineal para mezclarlos:

$$T(n) = 2T\left(\frac{n}{2}\right) + n, \quad T(1) = 0.$$

Se aplica el Teorema Maestro en la forma

$$T(n) = aT(n/b) + cn^d.$$

En este caso

$$a = 2, \quad b = 2, \quad c = 1, \quad d = 1.$$

Como

$$a = 2 = b^d = 2^1,$$

corresponde al caso $a = b^d$, para el cual

$$T(n) \in \mathcal{O}\left(n^d \log_b n\right) = \boxed{\mathcal{O}(n \lg n)}.$$

La sustitución permite ver también la expresión exacta del contador de mezcla:

$$\begin{aligned} T(n) &= 2T(n/2) + n, \\ &= 2[2T(n/4) + n/2] + n = 2^2T(n/2^2) + 2n, \\ &= 2^3T(n/2^3) + 3n, \\ &\vdots \\ &= 2^kT(n/2^k) + kn. \end{aligned}$$

La recursión finaliza cuando $n/2^k = 1$, de donde $k = \lg n$ y $2^k = n$. Como $T(1) = 0$,

$$T(n) = nT(1) + n \lg n = \boxed{n \lg n}.$$

Así, tanto el Teorema Maestro como la sustitución conducen a

$$\boxed{T(n) \in \mathcal{O}(n \lg n)}.$$

3.3. Evidencia experimental

Como prueba funcional, [38, 27, 43, 3, 9, 82, 10, 1] se transformó en [1, 3, 9, 10, 27, 38, 43, 82].

n	Operaciones de mezcla	Comparaciones	$n \lg n$
16	64	46	64
32	160	120	160
64	384	318	384
128	896	748	896
256	2048	1697	2048
512	4608	3773	4608
1024	10240	8177	10240

Cuadro 3: Resultados de Merge Sort.

Las comparaciones dependen de los datos; el contador de operaciones de mezcla es el que modela el término n de la recurrencia y coincide exactamente con $n \lg n$.

```
MERGE SORT RECURSIVO -  $O(n \lg n)$ 
=====
Ejemplo antes:  [38, 27, 43, 3, 9, 82, 10, 1]
Ejemplo despues: [1, 3, 9, 10, 27, 38, 43, 82]

n      Desordenado  Ordenado  Operaciones mezcla  Comparaciones  n*log2(n)
-----
16      Si          Si          64                  46              64
32      Si          Si          160                 120             160
64      Si          Si          384                 318             384
128     Si          Si          896                 748             896
256     Si          Si          2048                1697            2048
512     Si          Si          4608                3773            4608
1024    Si          Si          10240               8177            10240
=====
```

Figura 3: Ejecución de Merge Sort recursivo.

4. Complejidad $O(n^2)$: trabajo lineal por nivel

4.1. Implementación

En cada llamada se ejecutan n operaciones reales y luego se resuelve el problema $n - 1$.

```
void cuadratico(int n) {
    if (n <= 0) return;
    for (int i = 0; i < n; i++) contador++;
    cuadratico(n - 1);
}
```

4.2. Recurrencia y resolución

El ciclo ejecuta n operaciones y luego se procesa $n - 1$:

$$T(n) = T(n - 1) + n, \quad T(0) = 0.$$

Es una relación lineal no homogénea. La homogénea asociada es

$$T_h(n) = T_h(n - 1),$$

cuya ecuación característica es $r - 1 = 0$. Así, $r = 1$ y

$$T_h(n) = C.$$

Como el término no homogéneo es $F(n) = n = n(1)^n$ y 1 es raíz de multiplicidad uno, siguiendo el método visto en clase se propone

$$T_p(n) = n(An + B) = An^2 + Bn.$$

Sustituyendo en $T(n) - T(n - 1) = n$:

$$An^2 + Bn - [A(n - 1)^2 + B(n - 1)] = n.$$

Al desarrollar y agrupar términos:

$$(2A)n + (B - A) = n.$$

Igualando coeficientes,

$$2A = 1, \quad B - A = 0,$$

de donde $A = B = 1/2$. Por tanto,

$$T(n) = C + \frac{1}{2}n^2 + \frac{1}{2}n.$$

Con $T(0) = 0$ resulta $C = 0$ y

$$T(n) = \frac{1}{2}n^2 + \frac{1}{2}n = \frac{n(n+1)}{2}.$$

La misma expresión aparece al expandir

$$T(n) = n + (n - 1) + \cdots + 1 = \sum_{k=1}^n k.$$

El término dominante es $\frac{1}{2}n^2$, por lo que

$$T(n) \in \mathcal{O}(n^2).$$

4.3. Evidencia experimental

n	Operaciones	$n(n+1)/2$	Razón
10	55	55	—
20	210	210	3.8182
40	820	820	3.9048
80	3240	3240	3.9512
160	12880	12880	3.9753
320	51360	51360	3.9876
640	205120	205120	3.9938

Cuadro 4: Algoritmo cuadrático.

Al duplicar n , la razón se aproxima a $4 = 2^2$.

ALGORITMO RECURSIVO CUADRATICO - $O(n^2)$				
=====				
n	Operaciones	$n(n+1)/2$	Coincide	Razon

10	55	55	Si	-
20	210	210	Si	3.8182
40	820	820	Si	3.9048
80	3240	3240	Si	3.9512
160	12880	12880	Si	3.9753
320	51360	51360	Si	3.9876
640	205120	205120	Si	3.9938
=====				

Figura 4: Ejecución del algoritmo recursivo cuadrático.

5. Complejidad $O(n^3)$: trabajo cuadrático por nivel

5.1. Implementación

En cada llamada se ejecutan dos ciclos de tamaño n , es decir, n^2 operaciones, y luego se procesa $n - 1$.

```
void cubico(int n) {
    if (n <= 0) return;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) contador++;
    cubico(n - 1);
}
```

5.2. Recurrencia y resolución

En cada nivel se realizan n^2 operaciones y luego se procesa $n - 1$:

$$T(n) = T(n - 1) + n^2, \quad T(0) = 0.$$

La relación es lineal no homogénea. Su homogénea asociada,

$$T_h(n) = T_h(n - 1),$$

tiene ecuación característica $r - 1 = 0$, por lo que $T_h(n) = C$.

El término no homogéneo es $F(n) = n^2 = n^2(1)^n$. Como 1 es raíz de multiplicidad uno, el procedimiento estudiado conduce a una particular de la forma

$$T_p(n) = n(An^2 + Bn + D) = An^3 + Bn^2 + Dn.$$

Se sustituye en

$$T(n) - T(n - 1) = n^2.$$

Usando $(n - 1)^3 = n^3 - 3n^2 + 3n - 1$ y $(n - 1)^2 = n^2 - 2n + 1$, resulta

$$3An^2 + (-3A + 2B)n + (A - B + D) = n^2.$$

Al igualar coeficientes:

$$3A = 1, \quad -3A + 2B = 0, \quad A - B + D = 0.$$

Por tanto,

$$A = \frac{1}{3}, \quad B = \frac{1}{2}, \quad D = \frac{1}{6}.$$

Con $T(0) = 0$ la constante homogénea es cero, y se obtiene

$$T(n) = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n = \frac{n(n+1)(2n+1)}{6}.$$

Equivalentemente,

$$T(n) = \sum_{k=1}^n k^2.$$

El término dominante es $\frac{1}{3}n^3$; entonces

$$T(n) \in \mathcal{O}(n^3).$$

5.3. Evidencia experimental

n	Operaciones	$n(n+1)(2n+1)/6$	Razón
10	385	385	—
20	2870	2870	7.4545
40	22140	22140	7.7143
80	173880	173880	7.8537
160	1378160	1378160	7.9259
320	10973920	10973920	7.9627
640	87586240	87586240	7.9813

Cuadro 5: Algoritmo cúbico.

Al duplicar n , la razón se aproxima a $8 = 2^3$.

```
=====
ALGORITMO RECURSIVO CUBICO - O(n^3)
=====
```

n	Operaciones	n(n+1)(2n+1)/6	Coincide	Razon
10	385	385	Si	-
20	2870	2870	Si	7.4545
40	22140	22140	Si	7.7143
80	173880	173880	Si	7.8537
160	1378160	1378160	Si	7.9259
320	10973920	10973920	Si	7.9627
640	87586240	87586240	Si	7.9813

Figura 5: Ejecución del algoritmo recursivo cúbico.

6. Complejidad $\mathcal{O}(n^{10})$: trabajo n^9 por nivel

6.1. Implementación

Para ejecutar realmente n^9 operaciones sin escribir nueve ciclos manuales se utiliza una función auxiliar recursiva con nueve niveles.

```
void trabajoPotencia(int n, int nivel) {
    if (nivel == 9) { contador++; return; }
    for (int i = 0; i < n; i++) trabajoPotencia(n, nivel + 1);
}
void potenciaDiez(int n) {
    if (n <= 0) return;
    trabajoPotencia(n, 0); //  $n^9$  operaciones reales
    potenciaDiez(n - 1);
}
```

6.2. Recurrencia y resolución

El algoritmo ejecuta n^9 operaciones reales y después continúa con $n - 1$:

$$T(n) = T(n - 1) + n^9, \quad T(0) = 0.$$

Es una relación lineal no homogénea. La homogénea asociada

$$T_h(n) = T_h(n - 1)$$

tiene ecuación característica $r - 1 = 0$ y solución $T_h(n) = C$.

El término no homogéneo puede escribirse como

$$F(n) = n^9(1)^n.$$

Como $s = 1$ es raíz de multiplicidad $m = 1$ de la ecuación característica, el método visto en clase indica multiplicar por n^m la propuesta polinómica. Por ello se busca una particular de la forma

$$T_p(n) = n(A_9n^9 + A_8n^8 + \cdots + A_0),$$

que es un polinomio de grado 10.

Para determinar el orden asintótico basta identificar su coeficiente principal. Si el término dominante es An^{10} , entonces

$$T_p(n) - T_p(n - 1) = A [n^{10} - (n - 1)^{10}].$$

Por el binomio,

$$(n - 1)^{10} = n^{10} - 10n^9 + 45n^8 - \cdots,$$

de modo que el término principal de la diferencia es $10An^9$. Al compararlo con n^9 :

$$10A = 1 \implies A = \frac{1}{10}.$$

Así, la solución tiene como término dominante $\frac{1}{10}n^{10}$.

Por sustitución sucesiva también se obtiene

$$T(n) = n^9 + (n-1)^9 + \cdots + 1^9 = \sum_{k=1}^n k^9.$$

Además, como $k^9 \leq n^9$ para $1 \leq k \leq n$,

$$T(n) \leq \sum_{k=1}^n n^9 = n^{10}.$$

Por consiguiente,

$$T(n) \in \mathcal{O}(n^{10}).$$

6.3. Evidencia experimental

n	Operaciones	Incremento	n^9	$\sum_{k=1}^n k^9$
1	1	1	1	1
2	513	512	512	513
3	20196	19683	19683	20196
4	282340	262144	262144	282340
5	2235465	1953125	1953125	2235465
6	12313161	10077696	10077696	12313161

Cuadro 6: Algoritmo de complejidad $\mathcal{O}(n^{10})$.

La evidencia principal es $T(n) - T(n-1) = n^9$, verificada exactamente en cada fila.

```
=====
ALGORITMO RECURSIVO POTENCIA DIEZ - O(n^10)
=====
```

n	Operaciones	Incremento experimental	n^9	Sumatoria k^9	Coincide	Razon
1	1	1	1	1	Si	-
2	513	512	512	513	Si	513.0000
3	20196	19683	19683	20196	Si	39.3684
4	282340	262144	262144	282340	Si	13.9800
5	2235465	1953125	1953125	2235465	Si	7.9176
6	12313161	10077696	10077696	12313161	Si	5.5081

Figura 6: Ejecución del algoritmo recursivo de complejidad $\mathcal{O}(n^{10})$.

7. Complejidad $\mathcal{O}(n!)$: permutaciones

7.1. Implementación

El algoritmo fija un elemento en la posición actual, permuta recursivamente los restantes y restaura el arreglo para explorar la siguiente posibilidad.

```
void permutar(int[] a, int pos) {
    if (pos == a.length) { permutaciones++; return; }
    for (int i = pos; i < a.length; i++) {
```

```

        swap(a, pos, i);
        permutar(a, pos + 1);
        swap(a, pos, i);
    }
}

```

El contador principal registra permutaciones completas, no todas las instrucciones internas de Java.

7.2. Recurrencia y resolución

Sea $P(n)$ el número de permutaciones completas generadas. Para la primera posición existen n elecciones y, después de elegir una, quedan $n - 1$ elementos por permutar. Por ello,

$$P(n) = nP(n - 1), \quad P(1) = 1.$$

El coeficiente n no es constante; por tanto, esta recurrencia no corresponde al tipo lineal con coeficientes constantes ni a la forma del Teorema Maestro. Se resuelve mediante sustitución sucesiva:

$$\begin{aligned}
 P(n) &= nP(n - 1), \\
 &= n(n - 1)P(n - 2), \\
 &= n(n - 1)(n - 2)P(n - 3), \\
 &\vdots \\
 &= n(n - 1)(n - 2) \cdots 2P(1).
 \end{aligned}$$

Usando $P(1) = 1$,

$$P(n) = n(n - 1)(n - 2) \cdots 2 \cdot 1 = n!.$$

La misma estructura se observa en el árbol recursivo: hay n opciones en el primer nivel, $n - 1$ en el siguiente, y así sucesivamente, por lo que el número de hojas es $n!$. Cada hoja representa una permutación completa.

El contador utilizado mide precisamente esas permutaciones completas; la implementación realiza además intercambios, ciclos y llamadas recursivas. Para el crecimiento de los resultados generados se concluye

$$P(n) \in \mathcal{O}(n!).$$

7.3. Evidencia experimental

Para $n = 3$ se generaron las seis permutaciones esperadas. En las pruebas generales:

n	Generadas	$n!$	$P(n)/P(n-1)$
1	1	1	—
2	2	2	2
3	6	6	3
4	24	24	4
5	120	120	5
6	720	720	6
7	5040	5040	7
8	40320	40320	8
9	362880	362880	9

Cuadro 7: Generación recursiva de permutaciones.

La razón cumple exactamente $P(n)/P(n-1) = n$, como predice la recurrencia.

```
=====
GENERACION RECURSIVA DE PERMUTACIONES - 0(n!)
=====

Permutaciones generadas para [1, 2, 3]:
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 2, 1]
[3, 1, 2]
Total del ejemplo: 6

n      Permutaciones generadas      n! teorico      Coincide      Razon
-----
1      1                          1              Si             -
2      2                          2              Si             2.0000
3      6                          6              Si             3.0000
4      24                         24             Si             4.0000
5      120                        120            Si             5.0000
6      720                        720            Si             6.0000
7      5040                       5040           Si             7.0000
8      40320                       40320          Si             8.0000
9      362880                       362880         Si             9.0000
```

Figura 7: Ejecución de la generación recursiva de permutaciones.

8. Complejidad $\mathcal{O}(2^n)$: Torres de Hanoi

8.1. Implementación

Para mover n discos se mueven primero $n-1$ al auxiliar, luego el disco mayor y finalmente los $n-1$ discos al destino.

```
void hanoi(int n, char origen, char destino, char auxiliar) {
    if (n == 1) { movimientos++; return; }
    hanoi(n - 1, origen, auxiliar, destino);
    movimientos++;
    hanoi(n - 1, auxiliar, destino, origen);
}
```

8.2. Recurrencia y resolución

Para mover n discos se resuelve primero Hanoi con $n - 1$ discos, se mueve una vez el disco mayor y se vuelve a resolver Hanoi con $n - 1$ discos:

$$H(n) = 2H(n - 1) + 1, \quad H(1) = 1.$$

Esta relación lineal no homogénea puede resolverse por iteración, como en el ejemplo visto en clase:

$$\begin{aligned} H(n) &= 2H(n - 1) + 1, \\ &= 2^2H(n - 2) + 2 + 1, \\ &= 2^3H(n - 3) + 2^2 + 2 + 1, \\ &\vdots \\ &= 2^kH(n - k) + \sum_{i=0}^{k-1} 2^i. \end{aligned}$$

El caso base se alcanza cuando $n - k = 1$, es decir, $k = n - 1$. Entonces

$$H(n) = 2^{n-1}H(1) + \sum_{i=0}^{n-2} 2^i.$$

Como $H(1) = 1$ y la suma geométrica satisface

$$\sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1,$$

se obtiene

$$H(n) = 2^{n-1} + 2^{n-1} - 1 = \boxed{2^n - 1}.$$

También puede aplicarse el método de relaciones no homogéneas: la homogénea $H_h(n) = 2H_h(n - 1)$ tiene raíz $r = 2$ y solución $C2^n$. Para una particular constante A ,

$$A = 2A + 1 \implies A = -1.$$

Así, $H(n) = C2^n - 1$; usando $H(1) = 1$ se obtiene $C = 1$, nuevamente $H(n) = 2^n - 1$. Por tanto,

$$\boxed{H(n) \in \mathcal{O}(2^n)}.$$

8.3. Evidencia experimental

Para $n = 3$ se ejecutaron correctamente 7 movimientos. La tabla resume las pruebas principales:

n	Movimientos	$2H(n-1) + 1$	$2^n - 1$	Razón
1	1	—	1	—
2	3	3	3	3.0000
3	7	7	7	2.3333
4	15	15	15	2.1429
5	31	31	31	2.0667
6	63	63	63	2.0323
7	127	127	127	2.0159
8	255	255	255	2.0079
9	511	511	511	2.0039
10	1023	1023	1023	2.0020
15	32767	32767	32767	2.0001
20	1048575	1048575	1048575	2.0000

Cuadro 8: Torres de Hanoi.

La razón $H(n)/H(n-1)$ tiende a 2, coherente con el crecimiento exponencial de base 2.

```
=====
TORRES DE HANOI RECURSIVO - 0(2^n)
=====
```

Movimientos para n = 3:
Mover disco 1 de A a C
Mover disco 2 de A a B
Mover disco 1 de C a B
Mover disco 3 de A a C
Mover disco 1 de B a A
Mover disco 2 de B a C
Mover disco 1 de A a C
Total del ejemplo: 7

n	Movimientos reales	$2H(n-1)+1$	$2^n - 1$	Coincide	Razon
1	1	—	1	Si	—
2	3	3	3	Si	3.0000
3	7	7	7	Si	2.3333
4	15	15	15	Si	2.1429
5	31	31	31	Si	2.0667
6	63	63	63	Si	2.0323
7	127	127	127	Si	2.0159
8	255	255	255	Si	2.0079
9	511	511	511	Si	2.0039
10	1023	1023	1023	Si	2.0020
11	2047	2047	2047	Si	2.0010
12	4095	4095	4095	Si	2.0005
13	8191	8191	8191	Si	2.0002
14	16383	16383	16383	Si	2.0001
15	32767	32767	32767	Si	2.0001
16	65535	65535	65535	Si	2.0000
17	131071	131071	131071	Si	2.0000
18	262143	262143	262143	Si	2.0000
19	524287	524287	524287	Si	2.0000
20	1048575	1048575	1048575	Si	2.0000

Figura 8: Ejecución recursiva de las Torres de Hanoi.

9. Conclusiones

Las ocho implementaciones permitieron relacionar directamente el comportamiento de programas recursivos con ecuaciones de recurrencia. Los contadores experimentales reprodujeron las expresiones

teóricas esperadas: $2 \lg n + 1$, $2n - 2$, $n \lg n$, $n(n + 1)/2$, $n(n + 1)(2n + 1)/6$, $\sum_{k=1}^n k^9$, $n!$ y $2^n - 1$.

Los experimentos también muestran cómo pequeñas diferencias en la estructura recursiva producen órdenes de crecimiento muy distintos. Reducir el problema a la mitad genera comportamiento logarítmico; resolver dos mitades y combinar linealmente conduce a $n \lg n$; acumular trabajo polinómico por niveles produce grados superiores; y ramificaciones recursivas como permutaciones y Hanoi generan crecimiento factorial o exponencial. En todos los casos, la relación de recurrencia proporciona una descripción matemática coherente con los resultados obtenidos en Java.